

Created by Sir Tim Berners Lee HTML5 current version

Basic language used to create web pages

Page	_____
Date	_____

HTML (Hyper text ^{Predetermined} markup language)

↓
It only structure content

Relates link data information

↓ why
It uses tags to mark content and tell browser: →

- What is a heading, Paragraph
- where to show images, links, tables, forms etc

Purpose

- ↳ Create structure of web pages
- ↳ Acts as foundation of a website
- ↳ Display text, images, links, audio, video
- ↳ works with CSS Javascript

Document structure

<!DOCTYPE html> - Tells browser this is HTML5

<html> - Root element

< head> → metadata (title, CSS, meta tags) Not visible
SEO info

< title> This is my first web page </title>
Page title

</head>

<body> → Visible content, Page content, contain Text image links

<h2 heading </h2>

<p> paragraph </p>

</body>

</html>

<!DOCTYPE html>

<html>

<body>

<h1> The mark element </h1>

<p> DO not forget to buy ~~milk~~ <mark> milk </mark>
today </p>

</body>

</html>

HTML Headings

<h1> heading 1 </h1> — largest

<h2> heading 2 </h2>

<h6> heading 6 </h6> — smallest

used for SEO &
structure

we can customize size by :-

<h1 style="font-size: 60px;"> Big heading </h1>

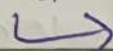
HTML Paragraphs

<p> This is a paragraph </p>

↳ Automatically starts a new line
Browser adds spacing.

linebreak

Ex Hello
 world



Break line without new
paragraph
empty tags

HTML tags → tell the browser how to display contents
 bold text

Types of Tags

1. Paired tags → come in pairs

<p> paragraph </p>

<h1> headings </h1> <u>
<i>

2. Empty tags → do not required to be closed

 <input>

<link>

Attributes → give extra information to tags

↓
Image
location

↓
width of
Image

↓
height of
Image

alt → alternate text

Markup tags

↳ special word such as variable URL
image block that tells browser
how to show content on web
page

<p> Helloworld </p>

↓
markup tags

browser understands
how text as a paragraph

HTML images

Empty tag contains list of attributes

mande
a.c

``

↓ ↓ ↓

image path alternate
+ text size

border $\rightarrow$ image border, thickness in terms of pixels

align $\rightarrow$ left, right/centre

→ Image name is always case sensitive.

for audio $\rightarrow$ mp3 is best format for compressed recorded music

HTML Examples

↳ Used to divide your browser window into multiple section

can load a separate HTML document

→ Collection of frames known as **frameset**

→ used to show another webpage inside your page.

Disadv

→ portable support

→ SGO issues

→ Back button problems

Ex `<iframe src="https://www.w3schools.com" title="W3Schools"></iframe>`

HTML tables → into rows / columns

`<tr>` → table rows

`<td>` data cell

`<th>` table heading.

`<table border="1">`

`<tr>`

`<th> Name </th>`

`<th> Age </th>`

`</tr>`

`<tr>`

`<td> Rahul </td>`

`<td> 20 </td>`

`</tr>`

`</table>`

HTML Comments `<!-- This is a comment -->`

Unordered Lists

 ↳ collection of related items that have no order or sequence

 item 1

 item 2

 item 3

Ordered lists / numbered lists

↳ order of items

 delhi

 mumbai

 Calcutta

Types

Attributes → Numeric Number (1, 2, 3)

Capital Roman Number (I II III) - uppercase

Small Roman Number (i ii iii) lowercase

Capital Alphabet (A B C)

small alphabet (a b c)

Ex `<ol type = "I">` → roman number uppercase

<code> HTML </code>	<u>output</u>
<code> Java </code>	I. HTML
<code> Javascript </code>	II. Java
<code> SQL </code>	III. Javascript
<code></code>	IV SQL

2. start specify where to start list items,

Ex `<ol type = "I" start = "5">`

<code> HTML </code>	<u>output</u>
<code> Java </code>	V -
<code> Javascript </code>	VI -
<code> SQL </code>	VII -
<code></code>	VIII -

3. LI (list item) → Represents list item in
↓
used inside

ordered unordered
and menu lists

`<ul>` and `<menu>`

↳ list items in bulletp

`<ol>`

↳ with numbers or Letters

Description lists

↳ list of terms, with a description of each term

<dl> → define list

<dt> term (name)

<dd> describe each term

ex

<dl>

<dt> coffee </dt>

<dd> black hot drink </dd>

<dt> milk </dt>

<dd> white cold drink </dd>

</dl>

Output

Coffee

- black hot drink

Milk

- white cold drink

Internal links

↳ Internal links are hyperlinks that direct the readers to a target page on your website

→ one page to another page on same domain

→ The menu bar on the your site includes internal links

External links

- ↳ used to interconnect two html web pages
- ↳ one domain to an entirely separate domain
- ↳ sometimes called outbound links
- ↳ can be created using anchor tags

Ex when you want to navigate to some other page of any website or any other URL by clicking on a link.

Internal

- same website
- fast
- Ex menu

External

- Different website
- Outbound
- Ex google

+ external
but

Anchor Element → used to create hyperlinks b/w source anchor and destination anchor

<a>

Anchor attributes

- href → specifies where the link will navigate to
 ↙ `<a href = "nextpage.html"> Click here </a>`
 link address
- target → links open in new tab or open in same window
 - blank → open in a new tab
 - parent → open in parent frame
 - self → open in same window
 - top → opens the same as self except with browser

Ex `<a href = "nextpage.html" target = "blank">`
The text will now be a link `</a>`

→ title → info about link
When we hover our mouse over a link with a title tag a little popup will display the text in this attribute

Ex `<a href = "nextpage.html" title = "Click here to see this link">` This text will now be link `</a>`

→ name / id → bookmarks
↳ used to create named anchor
↳ can create links to specific section on a page.

id attribute used instead

HTML form

↳ required when we want to collect some data from the site visitor

→ Take input from user & site visitor then post it on back-end application

such as CGI or PHP script

→ text fields, text area fields, ^{select one option from predefined list} dropdown menus, radio buttons, checkboxes

single line text

submit

send form data to the server

↓
when we want to select one option

↓
for multiple options




```
<form action = "server.php" method = "POST">
  <input type = "text" name = "username">
  <input type = "submit">
</form>
```

form attributes

- action → Backend script to process your passed data
- method → use to upload data Get post
- target → specify target window or frame where to show
blank, _self, _parent
- enctype → to specify how browser encodes the data before it sends to server.

Semantic HTML → uses meaningful tags that clearly describe what the content is
tells the browser, search engines, screen readers

```
<!DOCTYPE html>
```

```
<html>
```

```
<body> _____ page or section header
```

```
<header>
```

```
<h1> my blog </h1>
```

```
</header>
```

```

<nav> navigation menu
  <a href="#"> Home </a>
  <a href="#"> About </a>
</nav>
<main> main content
  <article> → independent content
    <h2> HTML Basics </h2>
    <p> HTML is the foundation of web
      development </p>
  </article>
</main>
<footer>
  <p> © 2026 My Blog </p>
</footer>
</body>
</html>

```

Non semantic HTML

↳ use tag which does not describe the meaning of content
→ only define layout or styling

<code><div></code>	<code></code>	<code></code>	<code><i></code>
↓	↓	↓	↓
Generic container	inline container	bold	italic text

<div>

<div> my Blog </div>

<div>

 Home

 About

</div>

Label → what an input field is for

eg <label for = "username"> Username: </label>

<input type = "text" id = "username">

→ always visible, improves accessibility

Placeholder - shows hint text inside the input field

→ Disappears when typing

eg. <input type = "text" placeholder = "Enter your username">

Javascript lightweight, cross platform

↳ client side scripting language
used to make web pages interactive
and dynamic

- Created by Netscape
- LiveWire → LiveScript → Javascript
- Runs inside the browser
- Interpreted language (runs line by line)
- Works with HTML and CSS
- Case sensitive

Client side	Server side
Runs in browser	Runs on server
Javascript	PHP, Python
Fast UI updates	Databases & logic

- Why → Easy to learn
- make web pages interactive
 - validates forms
 - Handles events (click, keypress)
 - Update content without reload

console.log → message on console
alert → browser open with an alert command
document.write → write insert message into HTML code

Embedding JS in HTML

Internal JS

```
<script>  
  alert ("Hello JS");  
</script>
```

Shows the JS
in working inside HTML

External JS

```
<script src="script.js">  
</script>
```

like Reusable and clean code
this connect
HTML with JS file

!DOCTYPE html

```
<html>  
<body>  
<script>  
  document.write("<h1> Hello world </h1>");  
</script>  
</body>  
</html>
```

JavaScript method used to
write content directly onto
the web page

represent
the web
page

write content on
the page

Event Driven Programming

↳ writing programs driven by user event
code runs only when an event occurs

- User action = event
- JavaScript function = reaction

user action → Event occurs → JavaScript execute

Ex `<button onclick = ('sayHello()')> Click me
</button>
<script>
function sayHello () {
 alert("Hello!");
}
</script>`

or

`<button onclick = "sayHello()"> Click </button>`

mouse events

→ onclick → Triggered when user clicks an element

`<button onclick = "alert('Clicked!')">
Click </button>`

→ onmouseover Triggered when mouse moves over an element

`<p onmouseover = "alert('mouse Over')"> Hover me
</p>`

→ onmouseout Triggered when mouse leaves the element

`<p onmouseout = "alert('mouse Out')"> move
away </p>`

Keyboard

- onkey down → Triggered when a key is pressed
`<input type="text" onkey down="alert('key pressed')">`
- onkey up → Triggered when key is released

Form Events

- onsubmit → Triggered when form is submitted
`<form onsubmit="return validate()">
 <input type="text">
 <button type="submit">submit</button>
</form>`

- onchange → when value changed
`<select onchange="alert('changed')">
 <option> HTML </option>
 <option> CSS </option>
</select>`

- onfocus → when input gets focus
`<input type="text" onfocus="alert('focus')">`
- onblur → when input loses focus

single Argument: $\rightarrow$ Ex function random(4) {

var randomno = Math.floor(Math.random()*4+1);
return randomno; }
(console.log(random(6)));

Page _____
Date _____

Window / Page Events

- onload $\rightarrow$ Triggered when page is fully loaded
`<body onload="alert('Page loaded')">`
- onresize $\rightarrow$ when window size changes
- onscroll $\rightarrow$ when page is scrolled

Javascript Function

Multi Argument

function myFunction(
width, height)

{
area = w * h;

return area;

console.log(area(10, 20));

}
alert("How are you?");

Reusable code, Executes on events,

Improves readability

DOM - Document Object Model

Bridge
b/w HTML
and JS

$\rightarrow$ HTML page converted into objects

Why and can be accessed and manipulated

- Read HTML element by JS

- Change text, styles

- React to user input

document $\rightarrow$ anchor $\rightarrow$ text
 $\rightarrow$ form $\rightarrow$ text area
 $\rightarrow$ link $\rightarrow$ radio
 $\rightarrow$ checkbox
 $\rightarrow$ select $\rightarrow$ option

Access Element

```
document.getElementById("output");
```

Change text

```
document.getElementById("output").innerHTML  
= "changed 1";
```

Change style

```
element.style.color = "red";
```



color, padding, bg color, border-top-width
font size, font family

hierarchy

- Window object
(Top of hierarchy)
outmost element

↓
Document object
(+ HTML document)
(+ browser window)
into a window
becomes document obj

↓
Form object
(= form
form object)

↓
form control elements
(objects + text fields)
(button, radio)

Methods or functions of document obj

write("string") → writes string on document

writeln("string") → write with new line
character at end

getElementById() → return element
having the given id value

ByName() → name value

ByTagName() → tag name

ByClassName() → class name

Variables in Javascript → number, string
Boolean, Array, object
null undefined

```
var name = "loovle";  
var age = 25;
```

type of age; // number

↳ datatype of datatype

↳ keywords used to declare variables in JS

var	let	const
mutable	mutable	immutable
Global / Function scoped	let age = 20;	Block scoped
var age = 20;	Block Scoped	const pi = 3.14;
Can be redeclared, reassigned	Cannot be redeclared, can be reassigned	Cannot be redeclared or reassigned
Ex var a = 10; ✓ var a = 20;	Ex let a = 10; a = 20; ✓ let a = 30; X	Value is fixed

(*) ⇒ `const arr = [1, 2, 3];`
`arr.push(4);` ✓
 ↳ Cannot change reference
 but can change contents

operators

== // value check

=== // value + type check

for ex "5" == 5 // true

"5" === 5 // false

Block scope let const

↳ variable declared inside a {} block cannot be accessed from outside the block

Local scope

↳ variables declared within a JavaScript function, are local to the function

Ex

```
function myfunction() {  
  let carName = "Volvo";  
}
```

Function Scope Variables defined inside a function are not accessible from outside the function

Ex

```
function myfunction() {  
  var carName = "Volvo";  
}
```

Global variable Accessible from any function declared outside the function.

Js Dialog boxes

→ Alert Dialog Box → used to give warning message to the user
when no validation

Ex

```
<script>
  alert("I am an alert box!");
</script>
```

Prompt Dialog Box - Pop up a text box to get user input enables to interact with the user method `prompt()`

Ex `var name = prompt("What is your name?");`
 → a label which you want to display in text box
 → a default string to display in text box

Confirm → ask for confirmation dialogue on particular action

`confirm()`

Ex `confirm("Select an option!");`

```
<script>
```

```
  var person = prompt("Please enter your name");
```

```
  document.write("You have entered: " + person);
```

```
</script>
```


To concatenate in JS - "+"

```
var message1 = prompt("1");
var message2 = prompt("2");
var message3 = prompt("3");
var message = message1 + message2 + message3 +
               message4;
console.log(message);
alert(message);
```

Object property → we can find property of object
and can change property of an object

ex To find length →

```
var phrase = "open source";
console.log(phrase.length);
console.log(phrase.toLowerCase());
console.log(phrase.toUpperCase());
```

Conversion

To convert into numbers from string

parseInt() → returns an integer

parseFloat() → returns a floating point number

Number() → Returns a number

ex

```
var msg1 = prompt("How many pencils?");
var msg2 = prompt("How many pens?");
Total = parseInt(msg1) + parseInt(msg2);
alert(Total);
```

math object mathematical tasks on numbers

→ math.method(number)

math.abs() → Return absolute number
 $(-1) \rightarrow 1$

math.ceil() → x rounded up
 $(1.2) \rightarrow 2$

math.floor(x) → x rounded down
 $(1.2) \Rightarrow 1$

math.round(x) → x rounded to nearest integer
 $(1.6) \rightarrow 2$

math.trunc(x) → Return integer part of x
 $(1.2) \rightarrow 1$

math.random() → To find random numbers

let $x = \text{math.random}()$;

let $x = \text{math.random()} * 20$;

let $x = \text{math.random()} * 100$;

let $x = \text{math.floor}(\text{math.random()} * 10 + 1)$;

if else $var a = 20;$
 $if (a \% 2 == 0) \{$
 $document.write("a is even number");$
 $\}$
 $else \{$
 $document.write("a is odd number");$
 $\}$

for loop $for (var i = 0; i < 10; i++) \{$
 $//$
 $\}$

while loop $while (i < 10) \{$
 $text += "The no is " + i;$
 $i++;$
 $\}$

do while $do \{$
 $text += "The number is "$
 $i++;$
 $\}$
 $while (i < 10);$

Array $\Rightarrow$ Storing a collection of multiple items under a single variable name

Ex WAP to print student list:

```
var student = ['shamu', 'dym', 'am', 'paul'];
for (var i = 0; i < student.length; i++)
{
    console.log(student[i]);
}
```

Method

Array.length()	Array.indexOf()
Array.reverse()	Array.of() $\rightarrow$ creates new array
Array.sort()	Array.slice() $\rightarrow$ Returns a copy of a portion
Array.fill()	
Array.join()	
Array.toString()	
Array.pop()	
Array.shift()	
Array.push()	
Array.unshift()	
Array.concat()	
Array.splice()	
Array.lastIndexOf()	

push method()

→ adding element to the end of an array

Ex `var arr = [1, 2, 3];`

`arr.push(4, 5);`

`console.log(arr);`

Output `[1, 2, 3, 4, 5]`

unshift method

→ add item at the beginning of our array.

Ex

Output `[4, 5, 1, 2, 3]`

pop() → Remove from last

Ex `arr = [1, 2, 3, 4];`

`array of numbers.pop();`

`console.log(array of no);`

Output `[1, 2, 3]`

shift Remove from beginning

Ex `[1, 2, 3, 4];`

`array of numbers.shift();`

`console.log(array of number)`

Output `[2, 3, 4]`

splice `arr.splice(start, deleteCount, item1, ...itemN)`

Ex `var primeNumbers = [2, 3, 5, 7, 9, 11];`

`primeNumbers.splice(4, 1, 13)`

`console.log(primeNumbers);`

// output `[2, 3, 5, 7, 13, 11]`

concat() merge two array
includes() checks if array contain specified
element or not

```
ex var students = [
    ['Amit', 'CS', '13'],
    ['Joe', 'ME', '19'],
    ['Jack', 'EE', '110']
];
```

```
(console.log(student[0][0])); // Amit
[0][1] // CS
```

→ function without name

→ Stored in a variable passed as an argument used in events, callbacks, array methods

```

Ex var greet - function() {
    alert ("Hello!");
};

```


Function with Parameters

```
var add = function(a, b) {  
    return a + b;  
};
```

Function in Events

```
<button onclick = "function() {alert('Clicked!')}";  
    Click me  
</button>
```

String var str = "JavaScript Programming";

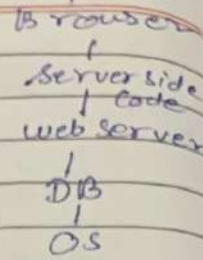
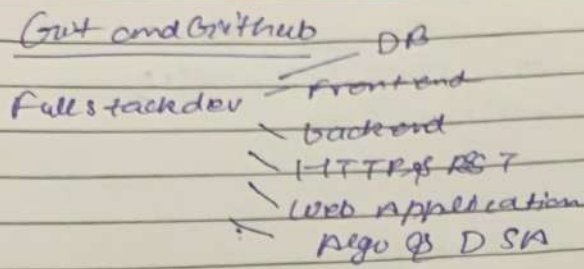
methods

str.length;

toUpperCase() → str.toUpperCase();

toLowerCase() → str.toLowerCase();

charAt() → str.charAt(0);



Version Control

↳ System that track changes in files over time so you can go back to older versions, compare changes, collaborate safely

Git

↳ distributed version control system

Track file changes supports branching and merging works offline

Fast & reliable

Store history

Git Repository Project folder + hidden .git folder

↳ A repository contains all files, folders and complete change history of project
for online storage of Git repositories.

- | | | |
|----------------------|----------------------------|-------------------------------|
| Git | maintained on local system | Github |
| → Software tool | | Cloud hosting platform |
| → Track version | | Store Git repositories online |
| → Works offline | | Require internet |
| → Command line based | | Web UI + Collaboration. |

Setup Git Environment

Step 1 Install Git

2. Create Git Repository → This creates .git folder to track project history
3. Add files → `git add file.txt`
`git add.`
4. Commit Files → It saves snapshot of project version permanently
5. Adding Folder in Git → `git add Folder Name`
`git commit -m "Added folder"`
6. Switch branch → `git checkout branch name`
↓
lets you move
between versions and branches of
project.

Ignore Files

• `gitignore` → tells git ~~which~~ which files/folders NOT to touch



→ ignore temporary, system or sensitive files

Git tools

- Git Bash
- GitHub Desktop
- Source Tree
- VS code Git integration

Workflow

1. Modify file
2. stage file → `git add`
3. commit → `git commit`
4. Push → `git push`

Commands

1. `git clone url` → copy
2. `git add files` → adds file contents
3. `git commit` → records a snapshot of staging area
4. `git status` → view the status of your file in working directory
5. `git diff` → shows diff what is staged and what is modified but unstaged
↳ a file ready to be saved